Typing the Harness: Quality and Knowledge Debt in LLM-Assisted MDE

Vasco Amaral
Universidade Nova de Lisboa
Lisbon, Portugal
alfonso.pierantonio@univaq.it

Miguel Goulão
Universidade Nova de Lisboa
Lisbon, Portugal
alfonso.pierantonio@univaq.it

Ha Phan
Toronto Metropolitan University
Toronto, Canada
alfonso.pierantonio@univaq.it

Alfonso Pierantonio
University of L'Aquila
L'Aquila, Italy
alfonso.pierantonio@univaq.it

Manuel Wimmer
University of L'Aquila
L'Aquila, Italy
alfonso.pierantonio@univaq.it

Abstract

Large language model (LLM) agents now write a large share of the code in some model-driven engineering (MDE) tools, under human direction and across many sessions. The apparatus that makes this work, the harness, is a set of markup documents that carry intent, context, and verdicts between a human director and two agents, an architect and an implementer. These documents are the control surface of the process, yet their roles, the quality factors they serve, and how well an LLM actually exploits them are left informal. We abstract from a 320k-line MDE workbench, `Jjodel`, a reference harness: a triad of executors, a taxonomy of document types, a process model, and a phased evolution in which the process periodically rewrites itself through recalibration. We then characterize the harness in two complementary ways. A quality goal model (GRL/NFR) records what each document type is meant to improve, and what it harms, exposing three structural trade-offs. A performance characterization maps each document type to the typical characteristics of LLMs (context-window cost, long-context recall, instruction following, lazy loading, determinism). Knowledge debt is the gap between the two: where the intended quality contribution is not realized. We measure that gap on the `Jjodel` corpus of 346 documents and outline a cross-model validation with three LLMs (Claude, Gemini, Mistral) visualized as radar diagrams. The contribution is the reference harness, its quality and LLM-performance characterization, and knowledge debt cast as a measurable intended-versus-realized gap.

Keywords

model-driven engineering, large language models, harness engineering, goal modelling, quality attributes, knowledge debt, agentic development

1 Introduction

A growing fraction of software, including MDE tooling, is now produced by LLM agents working under human direction. In the system we study the human author writes close to none of the code; the work is deciding what to build and maintaining the apparatus that lets agents build it reliably across sessions. That apparatus, the *harness*, is a set of markup documents that carry intent, context, and verdicts between the agents. These documents are not passive notes: they are the control surface of the process.

The difficulty is that this control surface is informal. Documents accumulate, drift from the code, and encode assumptions that live only in the director's head. The accidental complexity of writing code has shrunk, but the essential complexity of keeping knowledge faithful to the system has not [2]. We call the gap between the domain knowledge a document should incarnate and the knowledge actually internalized by its author or its consuming agents *knowledge debt*, by analogy with technical debt [4, 8] but distinct from it: knowledge debt is about what the artifacts fail to carry, not about the quality of the code.

This paper abstracts a reusable harness from a real, heavily LLM-assisted MDE project and characterizes it along two axes. First, what each document type is *meant* to achieve: which quality factors it improves and which it harms, captured as a goal model. Second, how each document type *actually* fares given the characteristics of LLMs: context-window pressure, recall over long context, instruction following, lazy loading, determinism. Knowledge debt is the divergence between the two. We model the process structure as a plain flow of typed documents and steps, an activity-and-artifact process model, and keep that model deliberately lightweight.

Research questions.

- **RQ1** What process and document types constitute a reusable harness for LLM-assisted MDE development?
- **RQ2** Which quality factors does each document type improve or harm, and what trade-offs result?
- **RQ3** How does each document type perform with respect to LLM characteristics, and where does the intended quality contribution diverge from the realized one (knowledge debt)?

Contributions. (1) A reference harness abstracted from a 320k-line case: a triad of executors, a document taxonomy, a process model, and a phased evolution with recalibration as a self-referential meta-transformation. (2) A quality goal model of the harness, with a contribution matrix and three structural trade-offs. (3) A characterization of document-type performance against LLM characteristics. (4) Knowledge debt reframed as a measurable intended-versus-realized gap, quantified on `Jjodel` and with a cross-model validation design (Claude, Gemini, Mistral) visualized as radar diagrams.

2 Background

LLM-based development has diversified into several styles that differ in how much autonomy the model is given, the unit of work

it acts on, and the persistent artifacts it leaves behind. We review them in roughly increasing order of autonomy.

LLM chats such as ChatGPT,[1] Gemini,[2] and Claude.ai[3] are general assistants: the developer describes a problem, the model returns code, and the developer pastes it back by hand. Nothing persists beyond the conversation, so the reasoning and the decisions live in an ephemeral session and are lost when it closes. This is where the project studied here began, cycling through several chat providers before settling on a structured loop.

LLM assistants of the GitHub Copilot[4] kind move inside the editor, offering inline completion that the developer accepts or rejects at the level of tokens and lines. The underlying code models were popularized by Codex [3], and empirical work reports substantial productivity effects alongside usability and trust caveats [10, 14]. The human stays in a tight review loop and little persists beyond the code itself.

LLM coding agents such as Claude Code[5] and OpenAI Codex[6] take a whole task and edit the code base across many steps, reading the repository, running builds and tests, and iterating from a terminal. Autonomy rises from a single suggestion to a multi-step change, and with it the need to constrain the agent and to record what it did, typically through instruction files the agent reads before acting.

Intelligent IDEs such as Cursor[7] and OpenCode[8] embed these agents in the editor, adding repository-wide context, multi-file edits, and persistent project rules files that steer the agent across sessions.

Spec-driven development, exemplified by Kiro[9] and the broader spec-driven impulse [5], inverts the flow: a structured specification (requirements, design, tasks) is authored first and drives generation, aiming at an auditable pipeline rather than ad hoc prompting.

Across these categories a common practice has emerged, largely independently: persistent instruction files that carry rules, conventions, and context to the model, from the constitution-like files coding agents read before each task to the rules and steering files of intelligent IDEs and spec-driven tools. This document layer is precisely what we call the harness. Yet a systematic review of LLMs for software engineering [7] shows the literature concentrating on the model producing code; the documents that coordinate the process, and whether they preserve domain knowledge, receive little attention. Table 1 situates the categories. The harness studied here is a directed multi-agent setting in which a human director coordinates an architect and an implementer through a typed document set; our object of study is that document set, not the code generation itself.

3 Problems of LLM-based development

Criticism of LLM-assisted development clusters around a few recognizable cores. We separate observable technical problems from the more structural epistemic and cognitive critiques, which are often the underestimated ones, and for each we note the lifecycle stage at which it surfaces (Table 2).

Two observations matter for the rest of the paper. First, the stage column shows that several problems are born at generation but become visible only later, at review, testing, or maintenance, which is exactly why a disciplined process with explicit review and recording steps can intercept them. Second, knowledge debt and cognitive atrophy stand apart: they are not defects in the code but in what the process fails to internalize, and unlike technical debt they escape any code-level check. This is the problem our harness is meant to expose and measure, and the one the rest of the paper operationalizes.

4 A reference harness for LLM-assisted development

We abstract the harness from `Jjodel`, a web-based MDE workbench (metamodelling, model editing, a transformation and an expression language, Ecore and XMI I/O) of about 320k lines, a large share produced by the implementer under direction.

4.1 The triad

The harness has three executors: a *director* (human) who authors intent and gates the loop, an *architect* (a conversational LLM) who reasons and generates prompts, and an *implementer* (a coding agent) who edits the code base. The director plays two roles the model keeps apart: *authoring* intent is an activity, while *gating* (accept, reject, redirect) produces no artifact and is a decision. We record the executor of each step (director, architect, implementer, or collaborative) and whether the step is automatic (performed by an agent with no human action) or manual.

4.2 Document taxonomy

Table 3 lists the generic document types abstracted from the corpus, with their role and their primary writer and reader. The writer and reader are read off the process model (the executors of the steps that produce and consume a document type), not stored on the document.

4.3 The process

Figure 1 shows the document-and-step structure and Figure 2 the enacted process. The running feature thread is: the director states intent, the architect produces a design and a discovery query, a discovery report comes back, the director gates on a risk and effort trade-off, the implementer changes the code, the change is committed, the log is appended. A step can consume and produce several document types (many-to-many), not just one.

4.4 Evolution and recalibration

The harness is not static. It evolved through four phases of LLM adoption (Figure 3): a spontaneous phase with a single chat and no documents; a three-agent phase introducing a persistent constitution; an intermediate phase adding handover documents for

[1] https://chat.openai.com
[2] https://gemini.google.com
[3] https://claude.ai
[4] https://github.com/features/copilot
[5] https://www.anthropic.com/claude-code
[6] https://openai.com/codex
[7] https://cursor.com
[8] https://opencode.ai
[9] https://kiro.dev

Table 1: Categories of LLM-based development, by the human role, the unit of work, the persistent artifacts produced, and where the domain knowledge resides. The last row is the setting studied here.

Category (examples)	Human role	Unit of work	Persistent artifacts	Where knowledge lives
LLM chat (ChatGPT, Gemini, Claude.ai)	prompter, copy-paste	snippet, file	ephemeral chat session	the session
LLM assistant, Copilot-like	reviewer of suggestions	token, line	none	model weights and code
LLM coding agent (Claude Code, Codex)	task-giver	task, feature	some (instructions, logs)	split between agent and code
Intelligent IDE (Cursor, OpenCode)	editor driver	multi-file edit	rules files, editor context	session and repository
Spec-driven development (Kiro)	specification author	spec to code	requirements, design, task docs	the specification
Directed multi-agent harness (this paper)	director who gates the loop	feature across roles	a typed document set	incarnated in the documents

Table 2: Typical problems of LLM-based development and the lifecycle stage at which each surfaces. Several originate at generation but only become visible later.

Problem	Surfaces at (lifecycle stage)	Characterization
Correctness and hallucination	generation; caught, if at all, at review and testing	Plausible but wrong code, invented APIs and library calls, no calibrated uncertainty; weak on concurrency, distributed systems, and non-local invariants that need global reasoning rather than pattern matching.
Security	generation; surfaces at build, testing, and in the supply chain	About 40% of generated programs carry vulnerabilities [9]; hallucinated package names enable slopsquatting [12]; agentic flows add prompt injection.
Code quality and technical debt	originates at generation; paid at maintenance and evolution	More short-term churn, more copy-paste than refactoring, erosion of DRY and of architectural coherence [6].
Knowledge debt and cognitive atrophy	cross-cutting; design and maintenance; the team	One-shot artifacts skip the understanding normally built through construction; de-skilling and loss of the productive struggle that forms competence. Distinct from technical debt, and the focus of this paper.
Productivity, perception vs reality	whole lifecycle; verification overhead at review and testing	Experienced developers on their own repositories were 19% slower with AI yet believed the opposite [1]; gains concentrate on boilerplate.
IP and provenance	generation; release; legal	Training on licensed code, verbatim regurgitation, opaque attribution and licensing.[10]
Homogenization and regression to the mean	design and generation	Convergence on dominant corpus patterns, anchoring on the first suggestion, loss of solution diversity.
Evaluation and benchmarks	assessment, cross-cutting	Toy benchmarks such as HumanEval [3], test-set contamination, and weak correlation with real software engineering [7].
Context and scale limits	design and maintenance	No coherent global model of large systems, inconsistency from one session to the next.

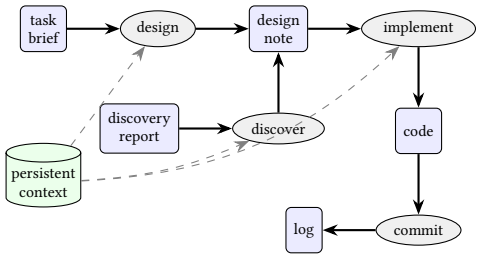

Figure 1: Document types (boxes) and the steps (ovals) that produce and consume them. Persistent context (cylinder) is read by many steps. The full harness has 11 document types and 13 steps.

design coherence; and the current phase of code-base support documents (discovery reports, session and prompt documents, decision and operational logs). Transitions are not drift but *recalibration* events in which the architect and implementer analyse the code base and the director gates a change to the process itself. Recalibration is therefore a self-referential meta-transformation, Recalibrate : $(CodeBase, Harness_n) \rightarrow Harness_{n+1}$, whose output is a new set of document types and steps for the process that runs it. The model in this paper is the current-phase snapshot; earlier document types such as handover carry a validity interval.

5 A quality goal model of the harness

The goal model (Figure 4) records, for each document type, the quality factors it is designed to improve or harm. Following the NFR framework and GRL, quality factors are softgoals and document

Table 3: Document taxonomy of the reference harness. Writer/reader are the executors of the producing/consuming steps. D director, A architect, I implementer.

Document type	Role	W	R
Persistent context	rules read before every task	A,D	A,I
Agent memory	distilled facts across sessions	A	A
Task brief	the director's intent	D	A
Design note	architect design and rationale	A	A,I
Discovery report	verdict on code-base state	A,I	D,A
Prompt artifact	instruction to the implementer	A	I
Specification	precise language or tool definition	A,D	A,I
Governance policy	conventions, standards, risk areas	D	A,I
Skill	lazily loaded competence	A,D	A,I
Operational log	append-only record	I	D,A
Handover or session	state passed between sessions	A	A
Code and changelog	the target and its record	I	A,I
Migration spec	realigns drifted artifacts	A	I

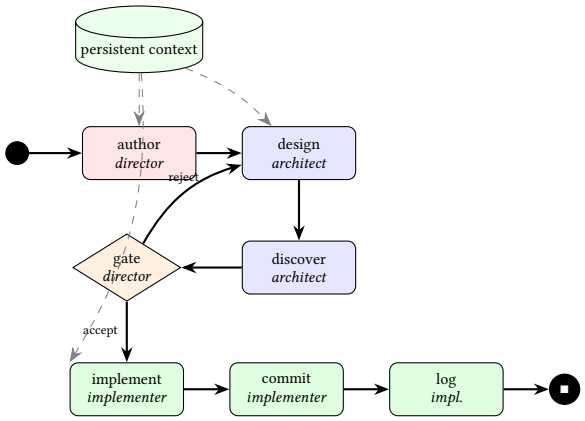

Figure 2: The enacted process. Activities are coloured by executor; the gate is a decision with a reject back-edge; persistent context feeds several activities by data flow (dashed).

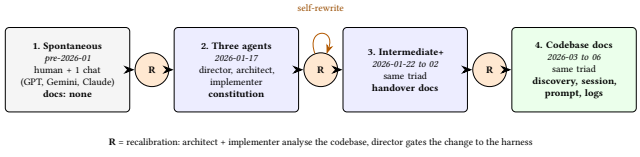

Figure 3: Harness evolution: four phases of LLM adoption with recalibration (R) as a self-referential meta-transformation. Dates are anchored to git history.

types are operationalizations contributing positively or negatively to them. Two top softgoals are in tension, knowledge fidelity (consistency, freshness, traceability, coverage, reproducibility, comprehensibility) and process efficiency (token economy, velocity), with governance (conformance, risk containment) cross-cutting. Table 4 is the full contribution matrix.

The matrix exposes three structural trade-offs. First, token economy versus knowledge fidelity: nearly every fidelity-improving document hurts token economy by competing for the context window or loading every session; the skill resolves this by loading lazily. Second, freshness versus consistency: persistent stores Make

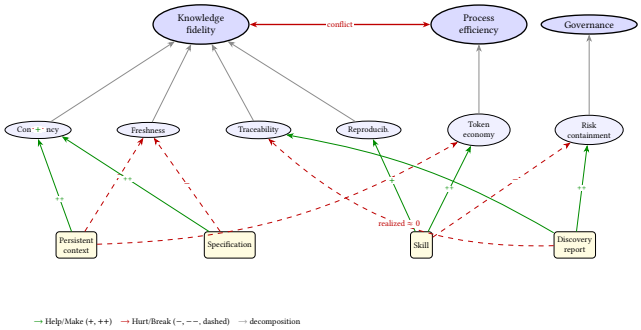

Figure 4: Softgoal interdependency graph of the harness. Document types (operationalizations) contribute Help/Make or Hurt/Break to quality factors. The three structural trade-offs and the discovery report's intended-versus-realized traceability gap are highlighted.

consistency but are themselves freshness liabilities as the code moves. Third, determinism versus efficiency: the skill Helps reproducibility yet Hurts risk containment because its selection is non-deterministic.

6 Document performance under LLM characteristics

The goal model is design intent. Whether a document realizes that intent depends on how an LLM consumes it. We characterize each document type against five LLM characteristics: context-window cost (how much it occupies), long-context recall (lost-in-the-middle exposure), instruction-following load, lazy-loadability, and determinism of use. Table 5 summarizes the salient interactions.

The characterization explains the trade-offs mechanically. The persistent context maximizes consistency but its high context-window cost and recall risk mean that as it grows, the probability that a given rule is actually attended to falls (lost-in-the-middle). The skill is the harness's response: it converts a high context-window cost into an on-demand load, at the price of a non-deterministic selection step. The operational log is cheap to consult on demand but expensive if kept unbounded in context, which is exactly where its stated rotation rule matters.

7 Knowledge debt as the intended-versus-realized gap

Knowledge debt is where a document's realized contribution falls short of its intended one. The goal model gives the intended contribution; an audit of the corpus gives the realized one; each negative realized link overriding a positive intended link is a debt instance. In process terms the gap surfaces as a conformance failure of the enacted process against its intended document-and-step model, of six kinds: a document that no type covers (missing document type); a real step that no defined step covers (missing step); a persistent store whose content no longer matches the code (stale context); a step whose output never appears as a written document (dead step);

Table 4: Contribution matrix: how each document type affects each quality factor. ++ Make, + Help, − Hurt, −− Break. Cons consistency, Fresh freshness, Trac traceability, Cov coverage, Repr reproducibility, Comp comprehensibility, Tok token economy, Vel velocity, Conf conformance, Risk risk containment.

Document type	Cons	Fresh	Trac	Cov	Repr	Comp	Tok	Vel	Conf	Risk
Persistent context	++	−		+	+	+	−−		+	+
Agent memory	+	−	+			+	−	+		
Task brief	+		+	+		+		+		
Design note	+		+	+		++	−			
Discovery report	+		+	+		+	+	+		++
Prompt artifact	+		+		+	+		+	+	+
Specification	++	−		+	+	++	−		+	
Governance policy	+			+	+	+	−−		++	+
Skill	+			+	+	+	++	+	+	−
Operational log			++	+			−−		+	
Handover or session	+	−	+			+	−	+		
Code and changelog			+	++						+
Migration spec	++	+			+				+	+

Table 5: Document performance under LLM characteristics. CW context-window cost, RC long-context recall risk, IF instruction-following load, LL lazy-loadable, DET determinism of use. H high, M medium, L low.

Document type	CW	RC	IF	LL	DET
Persistent context	H	H	H	no	H
Governance policy	H	M	H	part	H
Specification	H	H	M	part	H
Discovery report	M	M	L	yes	M
Skill	L	L	M	yes	L
Operational log	H	L	L	yes	H

Table 6: Measured indicators on `Jjodel` at a fixed commit. All static (computable from model and corpus).

Indicator	Value
Document typing coverage	≈67% (233/346)
Handoff incarnation rate	≈3.6% (7/193)
Persistent-context freshness	both stores now maintained
Governance-rule conformance	1 invariant violated 33×
Retired-step fraction	1 of 13 document types
Derived-doc drift	3–4% size of source spec
Type-authority presence	absent (metamodel file)

an activity reading a long-stale document (stale read); and a transient handoff carrying assumptions absent from every document type's definition (undocumented knowledge in transit).

We measured these on the `Jjodel` corpus of 346 authored documents at a fixed commit; every figure is scripted and re-runnable (Table 6). Two results are sharp. **Handoff incarnation.** Of 193 discovery reports, excluding the append-only log, only 7 (about 3.6 percent) are referenced by any downstream design, spec, or implementation document. The discovery report's intended traceability contribution is therefore realized only marginally: the knowledge stays in session. **Governance conformance.** The persistent context states a rotation invariant for the log (keep the last 20 entries, archive the rest); the active log holds 657 entries and no archive exists, a violation by a factor of about 33. The document asserts a rule the process does not keep.

8 Validation across models

The indicators above are static, computed on documents. A complementary question is whether document-type performance (Section 6) holds across LLMs. We outline a controlled protocol and visualize it as radar diagrams; data collection is ongoing and the values in Figure 5 are illustrative placeholders.

Protocol. For each document type, the same controlled task is given to three models, Claude, Gemini, and Mistral, on five measurable axes: instruction adherence (does the model follow the document), factual recall (does it retrieve facts the document states), output conformance (does its output obey the document), token efficiency (cost to achieve the above), and position robustness (does adherence survive when the document is placed late in a long context). Each axis is scored on a normalized scale by an objective check where possible. Three models is illustrative, not a definitive comparison.

9 Discussion and threats to validity

The study is n equals 1: one harness and one director, who is also the analyst, assisted by the same family of agents that run the harness. Generality is not claimed; the contribution is a reusable harness and a characterization method. Several indicators depend on classification choices a reviewer can contest (what counts as a downstream consumer, as cleanly typed); we report the rules so they can be reproduced. The goal-model contributions are first-pass and partly judgement-based; each strong link should be justified against evidence or revised. The cross-model validation is designed but not yet run, and three models are illustrative. The mechanism of skill selection is treated as inference over concepts rather than lexical match, supported by interpretability evidence [13]; the opposing probabilistic reading is also tenable and we do not adjudicate it.

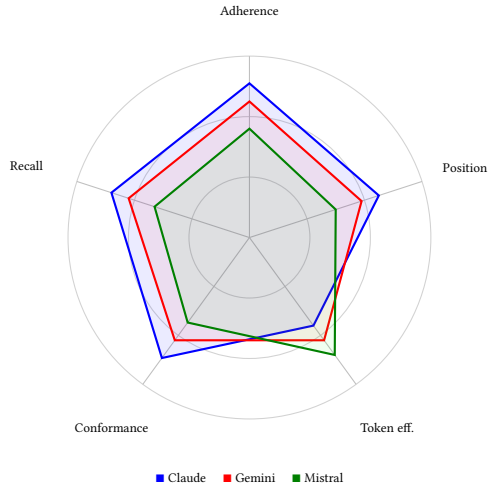

Figure 5: Illustrative radar of document-type performance across three models on the five protocol axes. Values are placeholders pending the measured run.

10 Related work

AI in software development. Work ranges from inline completion and its productivity studies [3, 10, 14] to agentic coding, surveyed broadly in [7]; the focus is the model producing code. We instead model the process and the documents that coordinate it. **Goal and quality modelling.** The NFR framework and GRL model softgoals and their contributions; we apply them to development-process documents rather than to a system under design. **Process modelling.** Strict workflows gave way to declarative process models [11] and to conformance checking and process mining over logs [15]; our process is director-driven yet internally non-deterministic, which points to property checking over traces. **Technical versus knowledge debt.** Technical debt is a property of the code [4, 8]; knowledge debt, as we use it, is a property of the documents and their conformance to a model of the process.

11 Conclusion and future work

We abstracted a reusable harness for LLM-assisted MDE development (RQ1), characterized each document type by the quality factors it serves and harms, with three trade-offs (RQ2), and mapped document types to LLM characteristics, defining knowledge debt as the intended-versus-realized gap and measuring it on a real corpus (RQ3). The reference harness, the quality and LLM-performance characterization, and the gap framing are the contribution. Future work completes the cross-model radar study, plots debt across phases rather than at one instant, and feeds the validated model back into a recalibration to generate the next harness.

References

[1] Joel Becker, Nate Rush, Beth Barnes, David Rein, et al. 2025. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.* Technical Report. METR. arXiv:2507.09089; 19% slowdown despite perceived speedup; verify authors.

[2] Frederick P. Brooks. 1987. *No Silver Bullet: Essence and Accidents of Software Engineering.* IEEE Computer. Essence versus accidental complexity; verify venue.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). Codex / HumanEval; verify author list.

[4] Ward Cunningham. 1992. The WyCash Portfolio Management System. *ACM SIGPLAN OOPS Messenger* 4, 2 (1992), 29–30.

[5] Martin Fowler. 2025. Specification-driven development (working note). Placeholder for the spec-driven framing; verify exact source.

[6] GitClear. 2024. *Coding on Copilot: Data Shows Downward Pressure on Code Quality.* Technical Report. GitClear. Copy-paste exceeds refactored lines; churn rises; industry report, verify.

[7] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* (2024). Verify volume/pages.

[8] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. 2012. Technical Debt: From Metaphor to Theory and Practice. *IEEE Software* 29, 6 (2012), 18–21.

[9] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *IEEE Symposium on Security and Privacy (S&P).* About 40% of generated programs vulnerable; arXiv:2108.09293.

[10] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590* (2023).

[11] Maja Pesic and Wil M. P. van der Aalst. 2006. A Declarative Approach for Flexible Business Processes Management. In *Business Process Management Workshops (Lecture Notes in Computer Science, Vol. 4103).* Springer, 169–180.

[12] Joseph Spracklen, Raveen Wijewickrama, Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *USENIX Security Symposium.* Package hallucination 5.2% (commercial) to 21.7% (open); slopsquatting; verify author list.

[13] Adly Templeton, Tom Conerly, Jonathan Marcus, et al. 2024. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.* Transformer Circuits Thread. Anthropic. Verify author list and URL.

[14] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA).*

[15] Wil M. P. van der Aalst. 2016. *Process Mining: Data Science in Action* (2 ed.). Springer.